```
class Dave
  def self.class_method_one
    puts "Class method one"
  end
  def Dave.class_method_two
    puts "Class method two"
  end
end

Dave.class_method_one
Dave.class_method_two
```

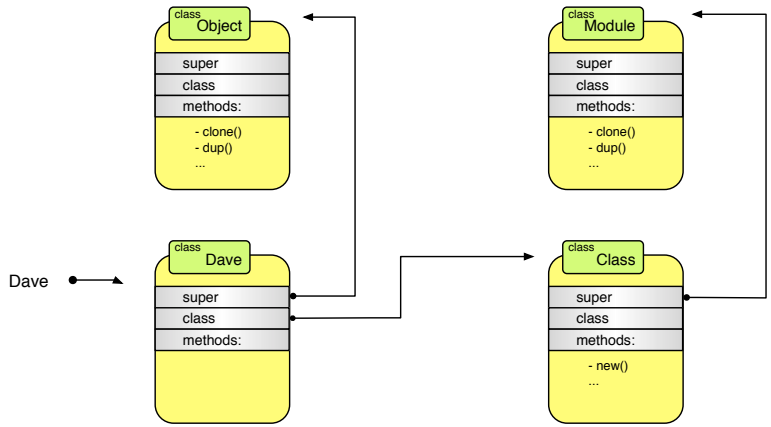
*produces:*

```
Class method one
Class method two
```

Now we can explain why the two forms are identical: inside the class definition, self is set to the class object Dave.

But now that we've looked at singleton methods, we also know that, in reality, there is no such thing as a class method in Ruby. Both of the previous definitions define singleton methods on the class object. As with all other singleton methods, we can then call them via the object (in this case, the class Dave).

Before we created the two singleton methods in class Dave, the class pointer in the class object pointed to class Class. (That's a confusing sentence. Another way of saying it is "Dave is a class, so the class of Dave is class Class," but that's pretty confusing, too.) The situation looks like this:




The object diagram for class Dave after the methods are defined looks like this: